

Manual de reglas.md

- Definir correctamente los tipos de datos del sistema.
- Separar estados, datos y comportamiento.
- Mantener tipado fuerte y arquitectura limpia.
- Evitar colisiones y errores por tipos inseguros.

Este archivo define las reglas fundamentales de diseño en C++ para el firmware del Zero (RP2040).

enum class (Estados y Comandos)

Define un conjunto cerrado de valores posibles.

Se utiliza para:

- Estados de máquina (Homing, Motor, Sistema).
- Comandos UART.
- Flags lógicos del sistema.

Ejemplo: Estado de Homing

```
enum class HomingStateXY
{
    INACTIVE,
    FIND_FIRST_EDGE_CW,
    FIND_SECOND_EDGE_CW,
    FIND_FIRST_EDGE_CCW,
    FIND_SECOND_EDGE_CCW,
    SEARCH_FAST_CW,
    SEARCH_FAST_CCW,
    REVERSE_EDGE_CW,
    REVERSE_EDGE_CCW,
    CALC_CENTER,
    MOVE_TO_CENTER,
    OK,
    ERROR
};
```

Uso:

```
HomingStateXY state = HomingStateXY::INACTIVE;
```

Los valores se utilizan con ::

Ejemplo: `HomingStateXY::OK`

Ventajas:

- No contamina el espacio global.
- No se convierte automáticamente a entero.
- No se mezcla con otros enums.
- Mayor seguridad de tipos.
- Código más claro y profesional.

Lo que NO puede hacer:

- No puede almacenar datos.
- No puede tener variables internas.
- No puede tener métodos.
- Solo define etiquetas posibles.

Ejemplo: Comandos UART

```
enum class Command
{
    STATUS,
    RESET_ERRORS,
    HOME_MOTOR1,
    HOME_MOTOR2,
    GET_ANGLE1,
    GET_ANGLE1_START,
    GET_ANGLE2,
    GET_ANGLE2_START,
    GET_ANGLE_STOP,
    UNKNOWN
};
```

Uso en `switch`:

```
switch(cmd)
{
    case Command::STATUS:
        break;

    case Command::HOME_MOTOR1:
        break;

    default:
        break;
}
```

struct (Contenedor de datos)

Se utiliza cuando necesitamos agrupar variables relacionadas.

Ideal para:

- Runtime de estados.
- Configuraciones.
- Estructuras de datos simples.

Ejemplo: Runtime de Homing

```
struct HomingRuntimeXY
{
    HomingStateXY state;
    unsigned long startTime;
    long firstEdge;
    long secondEdge;
    long centerPosition;
    bool fault;
};
```

Uso:

```
HomingRuntimeXY motor1;

motor1.state = HomingStateXY::INACTIVE;
motor1.firstEdge = 123;
motor1.fault = false;
```

`struct` es una caja de datos.

No protege información ni valida valores por defecto.

Ejemplo: Configuración

```
struct HomingConfig
{
    int microstepping;
    int reduction;
    int stepsPerRevolution;

    float fastSpeed;
    float slowSpeed;
    float acceleration;

    long steps90Deg;
    unsigned long timeout;
    int enablePin;
};
```

Es un bloque de configuración pasiva.

Perfecto para **struct** porque:

- Solo almacena datos.
- No contiene lógica.
- No necesita encapsulación.

➡ **class (Objeto con comportamiento)**

Se utiliza cuando:

- Necesitamos proteger datos (**private**).
- Hay validación interna.
- Existen métodos que operan sobre los datos.
- Hay lógica encapsulada.

🧩 Ejemplo conceptual

```
class Motor
{
private:
    int speed;

public:
    void setSpeed(int s)
    {
        if (s >= 0)
            speed = s;
    }

    int getSpeed() const
    {
        return speed;
    }
};
```

Aquí existe:

- Encapsulación.
- Validación.
- Lógica interna.

Eso justifica usar **class**.

🧠 Diferencia real entre struct y class

En C++ moderno la única diferencia real es:

- **struct** → miembros públicos por defecto.
- **class** → miembros privados por defecto.

Internamente funcionan igual.

La diferencia es intención y diseño.

Resumen conceptual

- 1 Estados y comandos → `enum class`
- 2 Contenedores simples de datos → `struct`
- 3 Objetos con comportamiento interno → `class`
- 4 Seguridad de tipos → siempre preferir `enum class`
- 5 Separar estado (`enum`) de datos (`struct`)

Con estas reglas el firmware Zero queda:

- Tipado fuerte.
- Modular.
- Escalable.
- Profesional.
- Fácil de mantener.